

Data 301 assignment 4

Angus Lindsay

2026-09-23

Exercise 1 part 1 Hyndman and Fan (1996) identified nine different definitions of sample quantiles that are commonly used in statistical software. These nine definitions can be divided into two broad groups: three discontinuous definitions based on rounding, corresponding to Types 1-3, and six continuous definitions based on linear interpolation, corresponding to Types 4-9. All nine definitions can be written in a common form. For an ordered sample $X_{(1)}$ less than or equal to $x_{(2)}$ less than or equal to $x_{(n)}$, the estimate of the p -th quantile is $Q_i(p) = (1 - \gamma)x_{(j)} + \gamma x_{(j+1)}$, where $j = \lfloor np + m \rfloor$, $g = np + m$, and m is a method-specific constant. For the discontinuous types, γ is either 0, 0.5, or depends on whether j is even, while for the continuous types one typically takes $\gamma = g$. Wicklin (2017) summarised these alternatives and showed that the differences among them are most noticeable for small samples, especially when adjacent order statistics are far apart. For larger samples, the nine definitions tend to produce very similar numerical results. The article also points back to Hyndman and Fan (1996), which remains the standard reference for understanding the motivation and properties of each definition.

If I had to choose one type of order statistic for this problem, I would use Type 8. The main reason is that Hyndman and Fan (1996) showed that Type 8 is approximately median-unbiased for population quantiles regardless of the underlying distribution. This is a distribution-free property, and it is especially useful when one cannot rely on large-sample normality. It is also the type explicitly recommended by Hyndman and Fan themselves. This matters here because the estimator `lamdaIQR` depends only on the first and third quartiles, so the quality of the quartile estimates directly determines the behaviour of the estimator. A quartile rule that is approximately median-unbiased should help reduce the bias of `lamdaIQR`, which is exactly the concern raised in the exercise. The advantage of Type 8 is likely to be most visible for small samples, such as $n = 5$ where the choice among quantile definitions has the greatest practical effect.

In my version of R, the default type of order statistics is Type 7, which is inherited from S for backwards compatibility. In Python, the default method in NumPy and SciPy, usually called linear, also corresponds to Type 7 of Hyndman and Fan. By contrast, SAS uses Type 2 as its default. Since the default in both R and Python is Type 7 rather than Type 8, one would need to specify the quantile type explicitly when implementing `lamdaIQR` if the aim is to use the Hyndman-Fan recommendation.

Exercise 1 part 2

```
# Bootstrap bias-corrected IQR estimator for exponential rate

lambda_iqr <- function(x, type = 8) {
  # IQR-based estimator of lambda for exponential distribution
  # Uses quartile type 8 by default (Hyndman & Fan recommendation)
  q1 <- quantile(x, 0.25, type = type)
  q3 <- quantile(x, 0.75, type = type)
  log(3) / (q3 - q1)
}

lambda_iqr_bc <- function(x, B = 200, type = 8) {
  # First-order bootstrap bias-corrected estimator
```

```

# x: observed sample
# B: number of bootstrap replications
# type: quantile typ

lambda_hat <- lambda_iqr(x, type = type)
n <- length(x)

# Bootstrap replicates
boot_ests <- numeric(B)
for (b in 1:B) {
  x_star <- sample(x, size = n, replace = TRUE)
  boot_ests[b] <- lambda_iqr(x_star, type = type)
}

# Bias estimate and correction
bias_star <- mean(boot_ests) - lambda_hat
lambda_tilde <- lambda_hat - bias_star # = 2*lambda_hat - mean(boot_ests)

return(list(lambda_hat = lambda_hat,
            lambda_tilde = lambda_tilde,
            bias_star = bias_star))
}

```

Exercise 1 part 3

```

set.seed(42)

lambda_ml <- function(x) {
  mean(x)^{-1}
}

run_monte_carlo <- function(lambda_true, n, R = 10000, B = 200) {
  est_ml <- numeric(R)
  est_iqr <- numeric(R)
  est_bc <- numeric(R)

  for (r in 1:R) {
    x <- rexp(n, rate = lambda_true)

    est_ml[r] <- lambda_ml(x)
    est_iqr[r] <- lambda_iqr(x, type = 8)

    bc <- lambda_iqr_bc(x, B = B, type = 8)
    est_bc[r] <- bc$lambda_tilde
  }

  # MSE for each estimator
  mse <- function(est, truth) mean((est - truth)^2)

  data.frame(
    lambda = lambda_true,
    n = n,
    MSE_ML = mse(est_ml, lambda_true),

```

```

    MSE_IQR = mse(est_iqr, lambda_true),
    MSE_BC  = mse(est_bc, lambda_true)
  )
}

# Run all configurations
results <- expand.grid(lambda = c(0.1, 1, 10), n = c(5, 30, 100))
mc_results <- do.call(rbind, lapply(1:nrow(results), function(i) {
  run_monte_carlo(results$lambda[i], results$n[i])
}))

print(mc_results)

```

```

##   lambda    n      MSE_ML      MSE_IQR      MSE_BC
## 1    0.1    5 5.622179e-03 9.189993e-03      Inf
## 2    1.0    5 5.916585e-01 9.893094e-01      Inf
## 3   10.0    5 5.869609e+01 9.300670e+01      Inf
## 4    0.1   30 3.819206e-04 9.273911e-04 1.130513e-03
## 5    1.0   30 3.866818e-02 9.552556e-02 1.180020e-01
## 6   10.0   30 3.977509e+00 9.659399e+00 1.176025e+01
## 7    0.1  100 1.041410e-04 2.330259e-04 2.742045e-04
## 8    1.0  100 1.062180e-02 2.347516e-02 2.758238e-02
## 9   10.0  100 1.038549e+00 2.384222e+00 2.797241e+00

```

```

mcse_mse <- function(est, truth, R) {
  sd((est - truth)^2) / sqrt(R)
}

```

For n equal to five, the bias-corrected mean square error is infinite for every λ . This happens because a bootstrap resample can contain five identical observations. Then the interquartile range is zero, so the IQR estimate of λ is infinite. One infinite bootstrap replicate makes the bootstrap mean infinite, so the first-order correction fails. This is a structural problem, not a simulation-size issue.

For n equal to thirty and n equal to one hundred, all mean square errors are finite. The maximum likelihood estimator has the smallest mean square error in every case. The IQR estimator has about two to two and a half times the mean square error of the maximum likelihood estimator. The bias-corrected estimator has an even larger mean square error than the uncorrected IQR estimator. So the ordering is consistently maximum likelihood, then IQR, then bias-corrected. The correction lowers bias but adds enough variance to raise the overall mean square error. Mean square errors also scale roughly with λ squared. Ten thousand replications are adequate for n equal to thirty and n equal to one hundred. For n equal to five, the infinite mean square error makes the replication count irrelevant unless the estimator itself is changed.

Excercise 2

```
library(fpp2)
```

```

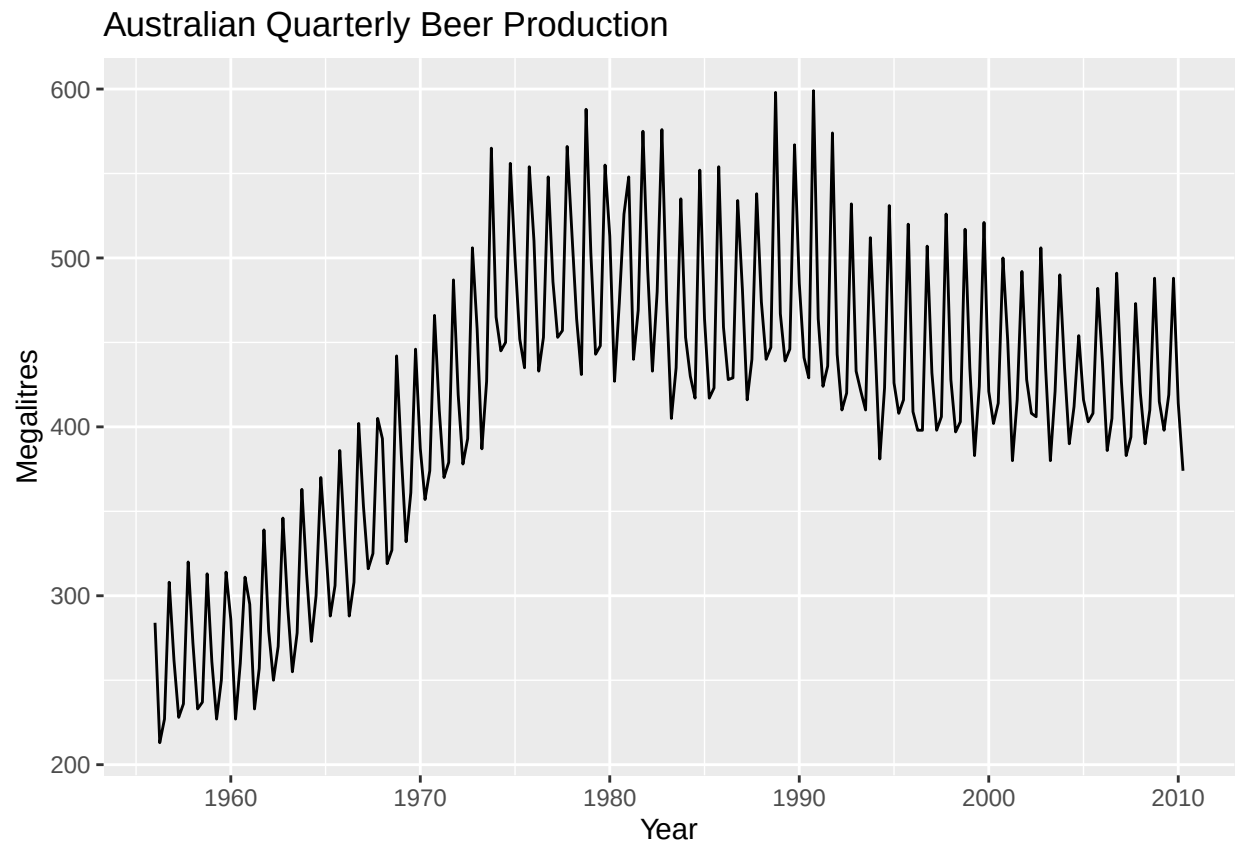
## -- Attaching packages ----- fpp2 2.5.1 --

## v ggplot2  4.0.3    v fma      2.5
## v forecast 9.0.2    v expsmooth 2.3

##

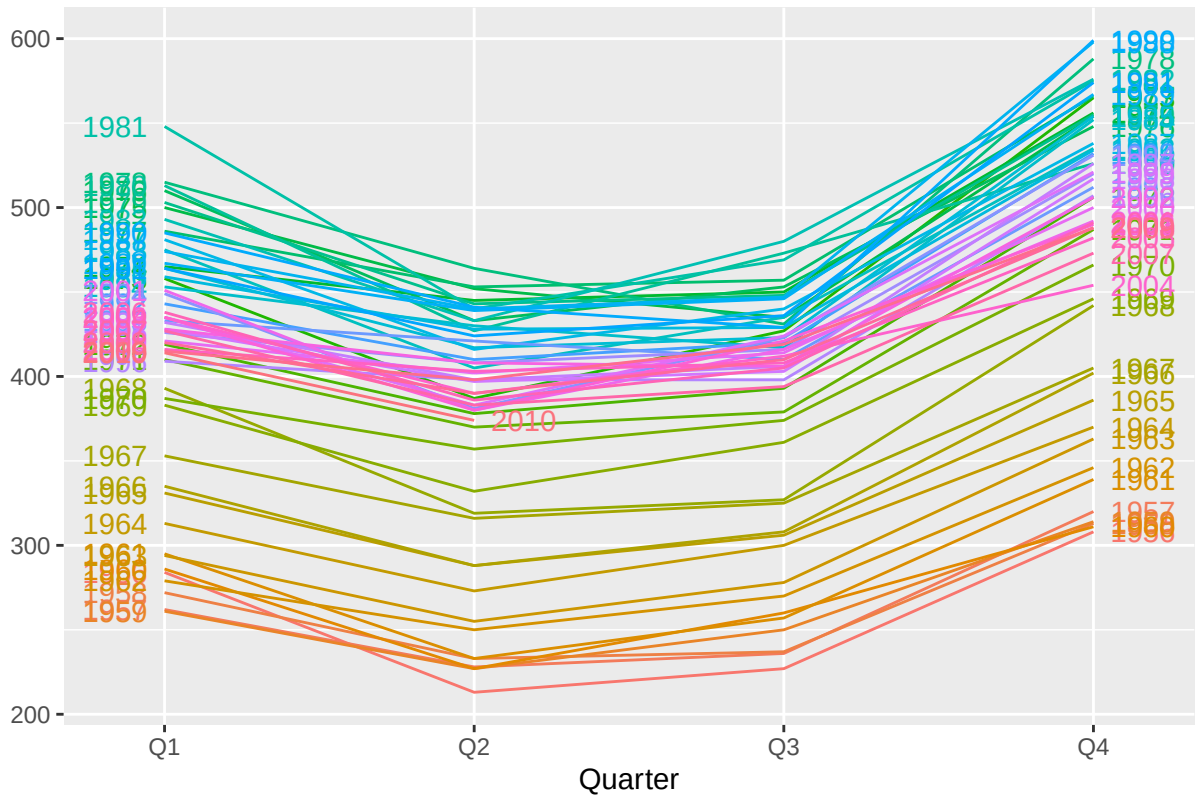
```

```
data(ausbeer)
autoplot(ausbeer) +
  ggtitle("Australian Quarterly Beer Production") +
  xlab("Year") + ylab("Megalitres")
```



```
ggseasonplot(ausbeer, year.labels = TRUE, year.labels.left = TRUE) +
  ggtitle("Seasonal Plot: Australian Beer Production")
```

Seasonal Plot: Australian Beer Production



1. Frequency: Quarterly, frequency = 4. The plot repeats every four observations per year.
2. Overall behaviour: Clear long-term trend: gradual rise to a peak around the late 1970s, then decline. Not linear.
3. Seasonality: Strong. Q4 and Q1 are highest; Q2 and Q3 are lowest. Pattern repeats each year.
4. Variability: Roughly constant in absolute terms, but seasonal swings look larger when production is high. A log transformation may help stabilize variance.
5. Unusual observations: No obvious data errors. The mid-1970s peak stands out. The sharp trend change after the late 1970s also attracts attention.
6. Seasonal plot: Shows the seasonal shape more clearly than the ordinary plot. It reveals that the seasonal amplitude changes over time—larger in high-production years, smaller later.
7. Seasonal subseries: Q4 has the highest typical values; Q2 has the lowest. The seasonal pattern is not perfectly stable; the gap between high and low quarters narrows in later years.
8. Detrend or transform? Yes. The series has trend and multiplicative-looking seasonality. A log transformation and possibly seasonal/first differencing are visually justified before modelling.